Data Science · Day 11 of 30 · about 60 minutes

Pandas II: filtering, sorting and grouping

🎯 Today's goal

By the end of this hour you will:

- **filter** a table: keep only the rows that match a condition
- combine conditions with `&` (and) and `|` (or), and use `isin` and `between`
- **sort** rows and pick the top few with `nlargest`
- **group** rows and summarise each group with `groupby`
- build a summary table with `pivot_table` and join two tables with `merge`

🤔 Why this matters

Yesterday you learned to load a table and look at it. But looking is not enough: real questions sound like *"Which orders were bigger than 8 items?"* or *"Which shop made the most money?"*. To answer them you **filter** (keep some rows), **sort** (put rows in order) and **group** (add up per shop, per product, per month).

Think of a big pile of receipts. Filtering is picking out only the coffee receipts. Sorting is putting them from biggest to smallest. Grouping is making one small pile per shop and writing the total on each pile. Almost every data analysis, and every report your boss will ever ask for, is made of these three moves.

☕ Today's data: a week of café sales

A small café chain has three shops: *Airport*, *Mall* and *Station*. Each row is one order line: on which date, in which shop, which product, how many (`qty` , short for quantity) and the price of one item. The data is **made up** for this lesson. As on Day 10, the CSV text is written straight into the code and read with `io.StringIO`. Then we add a `revenue` column (the money an order brought in: quantity × price).

```
import io
import pandas as pd

# Made-up sales of a small café chain (one week, three shops)
csv_text = """date,store,product,qty,price
2026-03-02,Airport,Coffee,8,3.5
2026-03-02,Airport,Sandwich,3,6.0
2026-03-02,Mall,Tea,4,2.5
2026-03-02,Mall,Cake,2,4.0
2026-03-02,Station,Coffee,6,3.5
2026-03-03,Airport,Juice,5,3.0
2026-03-03,Mall,Coffee,7,3.5
2026-03-03,Station,Sandwich,4,6.0
2026-03-03,Station,Tea,2,2.5
2026-03-04,Airport,Coffee,9,3.5
2026-03-04,Airport,Cake,3,4.0
2026-03-04,Mall,Juice,6,3.0
2026-03-04,Station,Coffee,5,3.5
2026-03-05,Airport,Tea,3,2.5
2026-03-05,Mall,Sandwich,5,6.0
2026-03-05,Mall,Coffee,4,3.5
2026-03-05,Station,Cake,6,4.0
2026-03-06,Airport,Sandwich,7,6.0
2026-03-06,Airport,Coffee,10,3.5
2026-03-06,Mall,Cake,5,4.0
2026-03-06,Station,Juice,3,3.0
2026-03-06,Station,Coffee,8,3.5
2026-03-07,Airport,Juice,4,3.0
2026-03-07,Mall,Coffee,9,3.5
2026-03-07,Mall,Tea,6,2.5
2026-03-07,Station,Sandwich,2,6.0
2026-03-08,Airport,Cake,4,4.0
2026-03-08,Mall,Sandwich,8,6.0
2026-03-08,Airport,Tea,5,2.5
2026-03-08,Station,Coffee,7,3.5
"""

sales = pd.read_csv(io.StringIO(csv_text))
sales["revenue"] = sales["qty"] * sales["price"]

print(sales.shape)
sales.head()
```

```
(30, 6)
```

	date	store	product	qty	price	revenue
0	2026-03-02	Airport	Coffee	8	3.5	28.0
1	2026-03-02	Airport	Sandwich	3	6.0	18.0
2	2026-03-02	Mall	Tea	4	2.5	10.0
3	2026-03-02	Mall	Cake	2	4.0	8.0
4	2026-03-02	Station	Coffee	6	3.5	21.0

30 rows and 6 columns. Now let's ask it questions.

🔍 Filtering: keeping only some rows

Filtering happens in two steps. First, a comparison on a column gives one `True` or `False` **per row**, just like comparing a NumPy array on Day 9. This list of True/False values is called a **mask** (or a *boolean filter*):

```
big = sales["qty"] >= 8     # True where the order had 8 or more items
big.head(10)
```

```
0     True
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9     True
Name: qty, dtype: bool
```

Second, you put the mask inside square brackets after the table. pandas keeps the rows where the mask is `True` and drops the others. Read `sales[sales["qty"] >= 8]` as *"sales, where qty is at least 8"*:

```
big_orders = sales[sales["qty"] >= 8]

print("Big orders:", len(big_orders), "of", len(sales))
big_orders
```

```
Big orders: 6 of 30
```

	date	store	product	qty	price	revenue
0	2026-03-02	Airport	Coffee	8	3.5	28.0
9	2026-03-04	Airport	Coffee	9	3.5	31.5
18	2026-03-06	Airport	Coffee	10	3.5	35.0
21	2026-03-06	Station	Coffee	8	3.5	28.0
23	2026-03-07	Mall	Coffee	9	3.5	31.5
27	2026-03-08	Mall	Sandwich	8	6.0	48.0

Notice the numbers on the left (the **index**): 0, 9, 18... The rows keep their original row numbers, so you can tell where they came from. This is where `.loc` and `.iloc` (Day 10) differ: `big_orders.iloc[1]` is the second row, and its label is 9, so it is the same row as `big_orders.loc[9]`. Filtering works with text columns too. Use `==` (two equals signs) to compare:

```
coffee = sales[sales["product"] == "Coffee"]

print("Coffee orders:", len(coffee))
print("Coffee revenue:", coffee["revenue"].sum())
```

```
Coffee orders: 10
Coffee revenue: 255.5
```

Filtering doesn't change the original. `sales` still has all 30 rows. The result is a new table, so store it in a new variable if you want to keep it.

🧩 Two conditions at once

What about "big orders *in the Mall*"? That's two conditions. In normal Python you'd write `and`, but on a whole column that doesn't work. Watch:

💡 This example has a mistake on purpose. Run it and read the error.

```
sales[(sales["store"] == "Mall") and (sales["qty"] >= 6)]
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     sales[(sales["store"] == "Mall") and (sales["qty"] >= 6)]
>           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
> ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),
> a.item(), a.any() or a.all().
> ```

"The truth value of a Series is ambiguous" means: `and` wants **one** True or False, but each side is a whole column of them, and Python doesn't know how to squeeze 30 answers into one. pandas has its own symbols that combine two masks **row by row**:

Symbol	Means	Keeps a row when…
`&`	and	both conditions are True
`|`	or	at least one condition is True
`~`	not	the condition is False

```
# Mall orders with 6 or more items: both must be True
mall_big = sales[(sales["store"] == "Mall") & (sales["qty"] >= 6)]
print(mall_big[["date", "product", "qty"]])

# Orders that are cake OR sold at the Station
is_cake = sales["product"] == "Cake"
at_station = sales["store"] == "Station"
cake_or_station = sales[is_cake | at_station]      # masks can be stored in variables
print("Cake or Station:", len(cake_or_station), "rows")

# Everything that is NOT coffee
not_coffee = sales[~(sales["product"] == "Coffee")]
print("Not coffee:", len(not_coffee), "rows")
```

```
          date   product  qty
6   2026-03-03    Coffee    7
11  2026-03-04     Juice    6
23  2026-03-07    Coffee    9
24  2026-03-07       Tea    6
27  2026-03-08  Sandwich    8
Cake or Station: 13 rows
Not coffee: 20 rows
```

Always put brackets around each condition when you use `&` or `|`: `(A) & (B)`. Without them Python reads the line in the wrong order and you get a confusing error.

Two shortcuts save a lot of typing:

- `.isin([...])`: is the value one of these? (instead of many `|`)
- `.between(low, high)`: is the number between low and high? (both ends included)

```
drinks = sales[sales["product"].isin(["Coffee", "Tea", "Juice"])]
print("Drink orders:", len(drinks))

medium = sales[sales["qty"].between(4, 6)]
print("Orders with 4 to 6 items:", len(medium))
```

```
Drink orders: 19
Orders with 4 to 6 items: 14
```

↕️ Sorting and the top few

You met `sort_values` on Day 10. It puts the rows in order of a column, smallest first. Add `ascending=False` for biggest first. You can also sort by two columns: first by the store name, then,

inside each store, by revenue. And because "give me the top 3" is so common, pandas has a shortcut: `nlargest(3, "column")` (and `nsmallest` for the bottom ones):

```
by_revenue = sales.sort_values("revenue", ascending=False)
print(by_revenue.head(3))
print()

# First by store (A to Z), then by revenue (biggest first) inside each store
by_store = sales.sort_values(["store", "revenue"], ascending=[True, False])
print(by_store.head(4))
print()

# The same top 3 as the first print, with less typing
print(sales.nlargest(3, "revenue"))
```

```
          date    store   product  qty  price  revenue
27  2026-03-08     Mall  Sandwich    8    6.0     48.0
17  2026-03-06  Airport  Sandwich    7    6.0     42.0
18  2026-03-06  Airport    Coffee   10    3.5     35.0

          date    store   product  qty  price  revenue
17  2026-03-06  Airport  Sandwich    7    6.0     42.0
18  2026-03-06  Airport    Coffee   10    3.5     35.0
9   2026-03-04  Airport    Coffee    9    3.5     31.5
0   2026-03-02  Airport    Coffee    8    3.5     28.0

          date    store   product  qty  price  revenue
27  2026-03-08     Mall  Sandwich    8    6.0     48.0
17  2026-03-06  Airport  Sandwich    7    6.0     42.0
18  2026-03-06  Airport    Coffee   10    3.5     35.0
```

🗂️ Grouping: one answer per group

"How much money did **each shop** make?" is a grouping question. With `groupby`, pandas does three things for you:

1. **split** the rows into groups (one pile per store),
2. **apply** a calculation to each pile (for example, sum the revenue),
3. **combine** the answers into one small result.

```
store_revenue = sales.groupby("store")["revenue"].sum()
store_revenue
```

```
store
Airport    229.5
Mall       219.0
Station    165.0
Name: revenue, dtype: float64
```

Read it as: *"group the sales by store, take the revenue column, and sum it"*. The result is a Series whose labels are the store names. Swap `sum` for `mean`, `count`, `min` or `max` to ask other questions. `.idxmax()` gives the *label* of the biggest value (like `argmax` on Day 9, but it returns the name instead of a position):

```
print(sales.groupby("store")["revenue"].mean())   # average order value
print()
print(sales.groupby("store").size())              # how many rows in each group
print()
print("Best store:", store_revenue.idxmax())
```

```
store
Airport    20.863636
Mall       21.900000
Station    18.333333
Name: revenue, dtype: float64

store
Airport    11
Mall       10
Station     9
dtype: int64

Best store: Airport
```

`size()` counts the rows in each group. (`count()` does almost the same, but it skips empty cells. Tomorrow you'll meet empty cells.)

Several answers at once with `agg`

`agg` (short for *aggregate*, which means "summarise many values into one") runs several calculations in one go:

```
sales.groupby("product")["qty"].agg(["sum", "mean", "max"])
```

product	sum	mean	max
Cake	20	4.000000	6
Coffee	73	7.300000	10
Juice	18	4.500000	6
Sandwich	29	4.833333	8
Tea	20	4.000000	6

With **named aggregations** you choose the name of each result column: `new_name=("column", "calculation")`. The store names end up as the index (the labels on the left). `reset_index()` turns them back into a normal column, which is handy for the next steps:

```
store_summary = sales.groupby("store").agg(
    orders=("qty", "count"),
    items=("qty", "sum"),
    revenue=("revenue", "sum"),
)
store_summary = store_summary.reset_index()
store_summary
```

	store	orders	items	revenue
0	Airport	11	61	229.5
1	Mall	10	56	219.0
2	Station	9	43	165.0

📊 Pivot tables: groups in two directions

You can group by two columns: `groupby(["store", "product"])`. The result is easier to read as a grid, with products down the side and stores across the top. That grid is a **pivot table** (you may know it from Excel):

```
pivot = pd.pivot_table(sales, values="revenue", index="product",
                       columns="store", aggfunc="sum", fill_value=0)
pivot
```

store product	Airport	Mall	Station
Cake	28.0	28.0	24.0
Coffee	94.5	70.0	91.0
Juice	27.0	18.0	9.0
Sandwich	60.0	78.0	36.0
Tea	20.0	25.0	5.0

`values` is what to add up, `index` becomes the rows, `columns` becomes the columns, and `aggfunc` is the calculation. `fill_value=0` writes 0 where a shop never sold a product. Now you can see at a glance that the Airport sells the most coffee.

🔗 Joining two tables with merge

Information often lives in separate tables. Our sales table doesn't say who manages each shop; a second (made-up) table does. `pd.merge` joins two tables by matching a shared column, called the **key**. Here the key is `store`. Notice that the managers table has a new *Harbour* shop with no sales yet:

```
managers = pd.DataFrame({
    "store": ["Airport", "Mall", "Station", "Harbour"],
    "manager": ["Omar", "Lina", "Sami", "Rana"],
})

# inner: keep only stores that appear in BOTH tables
inner = pd.merge(managers, store_summary, on="store", how="inner")
print(inner)
print()

# left: keep every row of the LEFT table (managers), fill gaps with NaN
left = pd.merge(managers, store_summary, on="store", how="left")
print(left)
```

```
     store manager  orders  items  revenue
0  Airport    Omar      11     61    229.5
1     Mall    Lina      10     56    219.0
2  Station    Sami       9     43    165.0

     store manager  orders  items  revenue
0  Airport    Omar    11.0   61.0    229.5
1     Mall    Lina    10.0   56.0    219.0
2  Station    Sami     9.0   43.0    165.0
3  Harbour    Rana     NaN    NaN      NaN
```

`NaN` means "Not a Number": an **empty cell**, because Harbour has no sales to join with. Use `how="inner"` when you only want matches, and `how="left"` when you must keep every row of the first table.

🧪 Putting it together

Real questions chain these steps. *"In the second half of the week (from 5 March), which product made the most money?"* Filter → group → sort:

```
# Dates written as year-month-day text can be compared like this
late_week = sales[sales["date"] >= "2026-03-05"]
product_revenue = late_week.groupby("product")["revenue"].sum()
product_revenue = product_revenue.sort_values(ascending=False)

print(product_revenue)
winner = product_revenue.index[0]          # the first label
print(f"Winner: {winner} with {product_revenue.iloc[0]:.2f} dollars")
```

```
product
Coffee      133.0
Sandwich    132.0
Cake         60.0
Tea          35.0
Juice        21.0
Name: revenue, dtype: float64
Winner: Coffee with 133.00 dollars
```

(Comparing the dates as text works here because they are written year-month-day. On Day 12 you'll turn text into real dates.)

⚠️ Common mistakes

- Using `and` / `or` between columns → `ValueError: The truth value of a Series is ambiguous`. Use `&` and `|`.
- Forgetting brackets around each condition: write `(sales["qty"] > 5) & (sales["store"] == "Mall")`.
- Writing `=` instead of `==` inside a filter. One `=` stores, two compare.
- Expecting the filter to change `sales`. It returns a new table; store it in a variable.
- Forgetting which column to summarise: `groupby("store").sum()` tries to add up *every* column, even the text ones. Pick one: `groupby("store")["revenue"].sum()`.

✅ Recap

- Filter: `df[df["col"] > 5]`. The comparison makes a True/False mask.
- Combine masks with `&` (and), `|` (or), `~` (not), with brackets around each one.
- `.isin([...])` and `.between(low, high)` are handy shortcuts.
- Sort: `sort_values("col", ascending=False)`; top rows: `nlargest(3, "col")`.
- Group: `df.groupby("a")["b"].sum()`; several results: `.agg(...)`; labels back to a column: `reset_index()`.
- `pd.pivot_table` makes a grid; `pd.merge(left, right, on="key", how="left")` joins two tables.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Big orders

Keep only the orders with **6 or more** items (`qty`). Store the filtered table in a variable called `big_orders`.

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})

# Create big_orders below
```

Hints:

1. First make the mask: `sales["qty"] >= 6`
2. Put the mask inside `sales[ ... ]` to keep only those rows.
3. `big_orders = sales[sales["qty"] >= 6]`

2. Top 3 orders

Find the **3 orders with the highest revenue**, biggest first. Store them in a variable called `top3` (a table with 3 rows).

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

# Create top3 below
```

Hints:

1. Two ways work: `nlargest`, or `sort_values` followed by `head(3)`.
2. `sales.nlargest(3, "revenue")`
3. `top3 = sales.sort_values("revenue", ascending=False).head(3)` works too.

3. Revenue per store

Calculate the **total revenue of each store** with `groupby` and store it in `store_totals` (a Series with one number per store). Then store the **name** of the best store in `best_store`. Find it with code, don't type the name.

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

# Create store_totals and best_store below
```

Hints:

1. Group by the store column, pick the revenue column, then sum: `sales.groupby("store")["revenue"].sum()`
2. `.idxmax()` on a Series gives the label of its biggest value.
3. `best_store = store_totals.idxmax()`

4. Two conditions

Create two filtered tables:

- `mall_big`: orders from the **Mall** store **and** with **more than 4** items. Use `&`.
- `tea_or_juice`: orders where the product is **Tea or Juice**. Use `|` or `isin`.

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})

# Create mall_big and tea_or_juice below
```

Hints:

1. Put brackets around each condition: `(sales["store"] == "Mall") & (sales["qty"] > 4)`
2. "More than 4" is `> 4`, not `>= 4`.
3. `tea_or_juice = sales[sales["product"].isin(["Tea", "Juice"])]`

5. Challenge: drinks or food?

The sales table doesn't say whether a product is a drink or food, but the `products` table does. Answer the question *"How much revenue came from drinks, and how much from food?"*:

1. Join the two tables on the `product` column with `pd.merge` (use `how="left"`) and store the result in `merged`.
2. Group `merged` by `category` and sum the revenue. Store the Series in `category_totals`.
3. Store the name of the category with more revenue in `best_category`, found with code.

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

products = pd.DataFrame({
    "product":  ["Coffee", "Tea", "Juice", "Cake", "Sandwich"],
    "category": ["drink", "drink", "drink", "food", "food"],
})

# 1. merged
# 2. category_totals
# 3. best_category
```

Hints:

1. `merged = pd.merge(sales, products, on="product", how="left")` adds a category column to every sale.
2. `category_totals = merged.groupby("category")["revenue"].sum()`
3. `best_category = category_totals.idxmax()`

➡️ Tomorrow

Cleaning messy data: today's data was perfectly tidy, but real data almost never is. Tomorrow you'll find and fix empty cells, duplicate rows, numbers stored as text, names spelled three different ways and impossible values like an age of 250.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Big orders

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})

big_orders = sales[sales["qty"] >= 6]
print(big_orders)
```

2. Top 3 orders

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

top3 = sales.nlargest(3, "revenue")
print(top3)
```

3. Revenue per store

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

store_totals = sales.groupby("store")["revenue"].sum()
best_store = store_totals.idxmax()
print(store_totals)
print("Best store:", best_store)
```

4. Two conditions

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})

mall_big = sales[(sales["store"] == "Mall") & (sales["qty"] > 4)]
tea_or_juice = sales[sales["product"].isin(["Tea", "Juice"])]
print(mall_big)
print(tea_or_juice)
```

5. Challenge: drinks or food?

```
import pandas as pd

# Made-up orders from one week
sales = pd.DataFrame({
    "store":   ["Airport", "Mall", "Station", "Mall", "Airport",
                "Station", "Mall", "Airport", "Mall", "Station"],
    "product": ["Coffee", "Tea", "Sandwich", "Coffee", "Cake",
                "Coffee", "Sandwich", "Juice", "Juice", "Tea"],
    "qty":     [8, 3, 6, 5, 2, 9, 7, 4, 6, 1],
    "price":   [3.5, 2.5, 6.0, 3.5, 4.0, 3.5, 6.0, 3.0, 3.0, 2.5],
})
sales["revenue"] = sales["qty"] * sales["price"]

products = pd.DataFrame({
    "product":  ["Coffee", "Tea", "Juice", "Cake", "Sandwich"],
    "category": ["drink", "drink", "drink", "food", "food"],
})

merged = pd.merge(sales, products, on="product", how="left")
category_totals = merged.groupby("category")["revenue"].sum()
best_category = category_totals.idxmax()
print(category_totals)
print("Best category:", best_category)
```